

PREFACE

General Purpose Hyper-Resilient Edge Supercomputing.

"In the abundance of water, the fool is thirsty." – Robert Nesta Marley, circa 1976

Marabunta started as a charitable High-Performance Computing (HPC) engine meant to give Médecins Sans Frontières (Doctors Without Borders) and other non-governmental organizations the supercomputing power required to predict Ebola outbreaks and manage responses to natural or man-made catastrophes, utilizing donor-provided computational FLOPs and algorithms.

Distributing massive AI and scientific workloads into hostile, infrastructure-poor environments required solving the hardest fundamental problems in distributed systems: bandwidth saturation, state synchronization, and execution trust.

Cambrian Radiation subsequently realized Marabunta's broader applications. Having solved those primitives to ensure the survivability of the network, a generic architecture naturally resulted from those emerging traits.

The system evolved into a substrate for executing any deterministic code, anywhere on Earth, with cryptographic proof of accuracy and physical location. Thus, it seems to have become commercially viable.

MARABUNTA REDONE

PREFACE: ASYMMETRIC DISTRIBUTED COMPUTE

General Purpose Hyper-Resilient Edge Supercomputing.

The Hyperscaler Paradigm is physically constrained by **Data Gravity** and **Egress Economics**. When processing massive datasets at the point of origin—or when executing logic within restricted, air-gapped jurisdictions—transmitting data to a centralized cluster violates the fundamental laws of network physics and sovereign economics.

Marabunta is an asymmetric execution substrate. It provides a sovereign alternative for environments where the data must remain at the origin.

The Marabunta Swarm is a decentralized mesh designed to coordinate arbitrary heterogeneous topologies. It inhabit high-density enterprise clusters, air-gapped corporate intranets, and consumer silicon alike, harnessing ambient thermodynamic dissipation to execute massive, deterministic workloads.

This document details the mechanical implementation of the core pillars required for sovereign edge compute: * **Physical Identity**: Deriving cryptographic proof of existence from CPU thermodynamic expenditure. * **Deterministic Sandboxing**: Executing untrusted code within strict memory and fuel boundaries. * **Segmented Routing**: Maintaining network consensus across firewalled and high-latency topologies.

This is the technical specification of a production-ready distributed engine.

```
# Illustrative: Monitoring local node integration with the global mesh
$ mrb swarm status --comprehensive
```

VOLUME 1: THE THERMODYNAMIC FOUNDATION

1.1 Identity in a Zero-Trust Vacuum (The Chrysalis Grinder)

Centralized Identity Management (IAM) is an architectural bottleneck in decentralized networks. Marabunta replaces the master server with a hardware-anchored identity system.

To participate in the Swarm, a node must bleed computational energy. During the initial boot phase, the node executes the **Chrysalis Grinder**, a memory-hard Proof-of-Work (PoW) algorithm. By saturating the L3 cache and the main memory bus using a unique hardware seed and core identifier, the node derives its sovereign **NodeId**.

Let E be the minimum thermodynamic energy expenditure required to discover a valid cryptographic nonce n , where D is the target network difficulty, L_{mem} is the latency of the main memory bus, and μ is the cache miss rate. The fundamental work equation is bounded by:

$$E \geq \left(\frac{2^{256}}{D} \right) \times \left(P_{idle} \cdot t_{hash} + P_{active} \cdot \mu \cdot L_{mem} \right)$$

This process mathematically neuters the efficiency of custom ASICs, which inherently possess low L_{mem} but suffer a massive μ penalty under this specific memory-hard algorithm. A Marabunta identity is a cryptographic receipt of physical work.

```
# Illustrative: Commencing hardware-anchored identity derivation
$ mrb identity derive --difficulty 0x000FFFF...
```

1.2 The Thermal Guillotine

Because Marabunta can execute untrusted code on consumer-grade hardware, it must assume environments where thermal dissipation is unmanaged. To prevent malicious algorithms or runaway simulations from causing physical damage to host silicon, the system implements the **Thermal Guillotine**.

The orchestrator continuously monitors physical CPU temperatures. Let $T_{core}(t)$ represent the internal silicon temperature at time t , and T_{crit} represent the absolute thermal ceiling (e.g., $91^{\circ}C$). The protection mechanism executes when the following threshold is breached:

$$\int_{t_0}^{t_1} \frac{dT_{core}}{dt} dt \geq T_{crit} \implies Interrupt(Epoch_{WASM})$$

If the threshold is breached—or if sensors are masked by a hypervisor—the daemon executes a hard interruption. By advancing the execution epoch of the underlying WebAssembly engine, the orchestrator terminates the process from outside its own execution boundary.

```
// Illustrative: Hardware health report during a sustained load event
GET /api/v1/node/thermal
{
  "state": "WARNING",
  "temperature": 88.2,
  "throttle_applied": true
}
```

1.3 The PARANOID Micro-Audit

High-priority tasks require consistent memory latency. To filter out weak hardware or virtualized environments that swap memory to disk, the orchestrator enforces a **Micro-Audit** (verify_mode: PARANOID).

The orchestrator demands the node allocate a 256MB matrix and execute a deterministic transformation within $2,500ms$. We utilize `black_box` compiler directives to ensure the OS cannot optimize the allocation away. Let τ_{alloc} be the time to allocate and $\tau_{transform}$ be the time to complete the deterministic hash matrix H_{matrix} . The condition for network acceptance is:

$$\tau_{alloc} + \tau_{transform}(H_{matrix}) \leq 2500ms$$

If the node fails this latency test, it is rejected from the high-performance queue. Only bare-metal hardware with high-speed physical RAM is permitted to claim sensitive payloads.

VOLUME 2: THE TOPOLOGY OF CHAOS

2.1 Statistical Kademlia Density Estimation

Marabunta strictly caps its local Kademlia representation to $k = 10,000$ peers to preserve host RAM. To accurately monitor a planetary-scale network without catastrophic memory exhaustion, the daemon utilizes **Statistical Kademlia Density Estimation**.

By measuring the mathematical XOR distance to its closest neighbors, the node probabilistically extrapolates the global topology. If b_{max} is the index of the deepest populated k-bucket (representing the longest shared binary prefix), the estimated global network size $\hat{N}$ is derived as:

$$\hat{N} = 2^{b_{max}} \times \left(\frac{k}{p_{collision}} \right)$$

The orchestrator maintains $O(\log N)$ routing efficiency and accurate global telemetry without holding the entire network state in memory.

```
// Illustrative: Fetching extrapolated Swarm topology
GET /api/v1/swarm/density
{
  "known_peers": 10000,
  "estimated_global_scale": 12500400,
  "confidence_interval": 0.95
}
```

2.2 Thermodynamic Gossip Jitter

Epidemic Gossip protocols suffer from “thundering herd” bottlenecks. To maintain thermodynamic equilibrium, Marabunta implements **Adaptive Gossip Jitter**.

The background daemon dynamically evaluates the extrapolated node count $\hat{N}$. As the network scales, the base heartbeat interval Δt aggressively decelerates, and a randomized exponential backoff J is injected.

$$\Delta t = \tau_{base} \times \lceil \log_{10}(\hat{N}) \rceil + J(0, \tau_{base})$$

This mathematically guarantees that the global packet rate remains bounded and asymptotic, preventing the 460-million packet/sec meltdown that would otherwise DDoS residential ISP backbones.

2.3 Instantaneous Work Resolution ($O(1)$ Data Gravity)

A traditional orchestrator scanning a million pending tasks performs an $O(N)$ linear search, pinning the CPU at 100% just looking for work. Marabunta completely decouples task tracking from execution loops.

The `KnowledgeStore` maintains a strict $O(1)$ indexed queue of unassigned chunks. When a 64-core enterprise node requests work, the CPU instantly pops contiguous task fragments off the stack in constant time. The CPU spends zero cycles scanning arrays, unlocking 100% of the silicon for the WebAssembly sandbox. # VOLUME 4: ENTERPRISE SOVEREIGNTY

4.1 The Corporate Intranet Illusion

Standard peer-to-peer (P2P) architectures assume a flat, universally routable public internet. When executing workloads across a midsize industry or university—where 99% of nodes reside behind strict corporate firewalls with zero inbound or outbound WAN access—traditional Kademlia DHTs instantly fail.

A sub-swarm of 500 air-gapped office workstations cannot download massive datasets from AWS S3, nor can they route computed results back to a global orchestrator. The network segments shatter into isolated enclaves.

To seamlessly integrate these high-density clusters, Marabunta introduces **Hierarchical Membrane Routing**.

4.2 The Egress Diode (Air-Gapped Proxying)

A designated edge node with authorized WAN access (e.g., an IT administrator's workstation) is promoted to a **Membrane Gateway**. This node acts as an **Egress Diode**, establishing an absolute one-way proxy between the corporate DMZ and the global Swarm.

When air-gapped internal workers attempt to stream the 1.5TB datalake via the Zero-Footprint WASI host-function, the Membrane Gateway intercepts the traffic. It natively spins up a lightweight HTTP Forward Proxy on a designated port.

Let B_{wan} be the external bandwidth of the Gateway, and $\sum B_{lan}$ be the aggregate consumption of the internal workers. The Gateway serves as a localized caching funnel, satisfying the condition:

$$\sum B_{lan} \leq B_{wan} \times Cache_{hit_rate}$$

Internal nodes safely pull global state without ever exposing the corporate LAN to external internet routing tables.

```
# Illustrative: Promoting a node to a Membrane Gateway
$ mrb network promote --role gateway --internal 192.168.1.100:9001
```

4.3 Hierarchical Back-Routing

When the internal corporate aggregator completes stitching together a 50MB wave-equation result, it cannot open a direct TCP socket to the global USP Orchestrator (O) due to firewall restrictions (**EHOSTUNREACH**).

Instead, the internal aggregator (n_i) encapsulates the payload into an **EgressRelay** message. The Membrane Gateway (G_m) unwraps this envelope, cryptographically signs it with its own identity, and forwards it to the global mesh. The routing trajectory becomes a Directed Acyclic Graph (DAG):

$$Route(n_i \circ O) = (n_i \xrightarrow{extLAN} G_m) \wedge (G_m \xrightarrow{extWAN} O)$$

This allows the Swarm to dynamically nest infinite layers of isolated sub-swarms, aggregating petabytes of compute power without compromising enterprise security postures.

```
// Illustrative: Membrane Gateway relay telemetry
GET /api/v1/membrane/telemetry
{
  "active_tunnels": 499,
  "bytes_exfiltrated": 52428800,
  "dropped_packets": 0
}
```

VOLUME 5: THE CRYPTOGRAPHIC ECONOMY

5.1 The Ledger of Sovereignty

A distributed execution engine requires an underlying economic mechanism to incentivize the donation of idle compute cycles. However, issuing digital credits using localized databases is inherently insecure. A malicious actor with filesystem access could arbitrarily modify their local SQLite ledger to inflate their balance.

Marabunta mathematically anchors its micro-credit system (MMX) to the cryptographic execution trace of the physics solver.

5.2 Ed25519 Replay Immunity

To prevent database tampering and receipt forgery, every MMX settlement is explicitly signed by the Orchestrator's `NodeIdentity`.

To prevent Replay Attacks—where a worker exports a valid 10 MMX receipt and duplicates it 1,000 times to inflate their tax deduction—the Orchestrator injects a strictly monotonic UNIX timestamp into the signature payload. The signature σ is an `Ed25519` function over the concatenated byte-array of the chunk ID (C_{id}), worker ID (W_{id}), credit amount (A), and timestamp (T):

$$\sigma = \text{Sign}_{\text{Ed25519}} \left(sk_{node}, H_{\text{Blake3}}(C_{id} \parallel W_{id} \parallel A \parallel T) \right)$$

When a tax authority or corporate auditor verifies the exported CSV, the cryptographic script guarantees that every single row is mathematically distinct and bound to a specific quantum of time.

```
# Illustrative: Exporting verifiable MMX cryptographic receipts  
$ mrb economy export --format csv --signed
```

5.3 The 10M TPS Wall (SQLite Batching)

While SQLite provides robust local storage, a regional hub settling thousands of transactions per second for a 10-million node swarm will instantly saturate the host operating system's thread pool, deadlocking the daemon due to file locks.

Marabunta bypasses physical NVMe IOPS limits by enforcing `PRAGMA journal_mode=WAL` (Write-Ahead Logging) and `wal_autocheckpoint=1000`. Settlement requests are pushed into a lock-free MPSC (Multi-Producer, Single-Consumer) background channel. A dedicated writer thread opportunistically batches up to 100 settlements into a single atomic SQLite transaction. This transforms 10,000 blocking disk I/O operations into 100 streamlined sequential writes, achieving massive throughput without async starvation.

ANNEX A: THE DEVELOPER EXPERIENCE

To ensure frictionless adoption by enterprise engineering teams, Marabunta provides a comprehensive, cloud-native developer experience.

- **The Compiler Toolchain:** Natively transpiles C++, Rust, and Python payloads into optimized WebAssembly (`wasm32-wasi`) binaries, automatically injecting the required Marabunta memory bounds and streaming host-functions.
- **IDE Integration:** Dedicated VSCode and IntelliJ plugins allow researchers to execute `0(1)` local sandbox testing and push payloads directly to the Kademlia DHT from their editor.
- **CI/CD Pipelines:** Native GitHub Actions and GitLab CI integrations for automated job submission, deterministic execution validation, and asynchronous artifact fetching.

```
# Illustrative: Compiling and deploying a job via the CLI toolchain
$ mrb build ./fwi_solver --target wasm32-wasi
$ mrb deploy ./fwi_solver.wasm --priority high
```

ANNEX B: THE SOVEREIGN ANALYTICS ENGINE

Operating a swarm of 10 million nodes requires unprecedented observability. The Marabunta Management Dashboard (The Panopticon) visualizes macroscopic network physics without crashing the observer's hardware.

- **Lazy DOM Rendering:** The backend utilizes lazy Kademia iterators, while the frontend employs strict 500-element FIFO ring-buffers. The UI guarantees a flat memory footprint, preventing browser Out-Of-Memory (OOM) crashes even when processing 5,000 telemetry events per second.
- **Constellation Telemetry:** WebGL and D3.js force-directed graphs render the topology of macro-coalitions and shifting economic treaties in real-time, aggressively culling edge matrices to preserve GPU rasterizers.
- **Ledger Auditing:** Real-time visibility into the MMX SQLite database via paginated endpoints, providing instantaneous compliance and economic tracking.

ANNEX C: PLANETARY DEPLOYMENT

Marabunta is designed as a frictionless contagion, capable of organically infiltrating diverse infrastructures.

- **Bare Metal & Residential:** Lightweight, zero-dependency binaries packaged as `.msi`, `.deb`, and `.rpm` for seamless installation on consumer gaming PCs and library desktops.
- **Academic Grids:** Multi-architecture containerization (Docker, Podman) for effortless scaling across university HPC clusters.
- **Enterprise Kubernetes:** Native Helm charts and Kubernetes Operators for highly available, auto-updating deployments within corporate DMZs.

ANNEX D: WAR GAMES

Marabunta is mathematically hardened against the physical realities of the public internet. Our architecture is empirically validated through continuous Chaos Engineering.

- **The Mantis Flight Data Recorder:** Constant WASI snapshotting enables deterministic Instruction-Set Replay. If a node in Brazil crashes during a massive simulation, the exact CPU state and memory boundaries can be perfectly reconstructed and debugged offline.
- **Network Partitions:** We routinely inject synthetic latency, simulated BGP hijacks, and split-brain geographic partitions into the control plane to mathematically prove the Swarm's ability to self-heal and re-route without centralized coordination.

```
# Illustrative: Injecting synthetic split-brain chaos into the local node
$ mrb chaos inject partition --duration 300s --drop-rate 0.99
```

ANNEX E: COMING SOON...

- **Distributed DiLoCo (LLM Training):** Being tested.
- **Sealed Computing (FHE):** Being developed.
- **IBM-CICS (COBOL, Assembly etc. w/ 2-phase commit):** Being tested.